

An Agentic Intelligent Code Refactoring Assistant for Legacy Systems

R26-SE-008

Project Proposal Report

Peiris M.I.A

IT22124180

B.Sc. (Hons) Degree in Information Technology Specialized in
Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

Code Understanding & Quality Assessment Agent

R26-SE-008

Project Proposal Report

Peiris M.I.A

IT22124180

B.Sc. (Hons) Degree in Information Technology Specialized in
Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Peiris M.I.A	IT22124180	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Dr. Kalpani Manathunga	17/03 /26	
Mr. Jeewaka Perera	13 - 03 - 2026	

ABSTRACT

Legacy software systems accumulate architectural degradation, technical debt, and structural complexity over prolonged evolution, making maintenance and refactoring increasingly risky and resource-intensive. Developers often struggle with deeply coupled modules, undocumented dependencies, and high cyclomatic complexity, leading to elevated cognitive load and increased probability of regression faults during modernization efforts. Although widely used static analysis platforms such as SonarQube and JDeodorant detect code smells and compute quality metrics, they primarily generate human-readable dashboards and rely on rule-based detection. These systems do not provide structured, machine-interpretable program representations nor refactoring-aware severity prioritization suitable for autonomous transformation pipelines.

This research addresses the gap by proposing an Intelligent Agent-Based Code Understanding and Quality Assessment System designed specifically for legacy system refactoring. The proposed agent performs deep structural analysis through Abstract Syntax Tree (AST) generation, control flow graph (CFG) construction, inter-module dependency modeling, and maintainability metric computation. It further applies contextual severity reasoning to detected design issues and exports structured knowledge representations in machine-readable formats to enable downstream automated refactoring agents.

The system will be implemented using language parsers (e.g., Tree-sitter), graph modeling frameworks, and JSON-based knowledge schemas. The expected outcomes include improved refactoring readiness, reduced manual comprehension time, structured quality intelligence for automation, and enhanced reliability in modernization processes.

Socially, the system reduces developer stress and improves software reliability for end users. Commercially, it enables cost-effective legacy modernization and supports scalable adoption of intelligent software engineering practices.

Keywords: Legacy Systems, Static Code Analysis, Code Smells, Agent-Based Software Engineering, Refactoring Automation

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our research supervisor for their continuous guidance, encouragement, and valuable insights throughout the preparation of this research proposal. Their expertise and constructive feedback greatly helped us in refining our research idea and shaping the direction of this study.

We also extend our appreciation to our co-supervisor for their support, suggestions, and technical advice which contributed significantly to improving the quality of this proposal.

Our heartfelt thanks go to the academic staff of the Faculty of Computing for providing the knowledge, resources, and academic environment necessary to conduct this research work.

Finally, this research proposal would not have been possible without the support and guidance of all those who contributed directly and indirectly to its completion.

TABLE OF CONTENT

DECLARATION.....	3
ABSTRACT.....	4
ACKNOWLEDGEMENT	5
TABLE OF CONTENT	6
LIST OF TABLES	6
LIST OF FIGURES	8
LIST OF ABBREVIATIONS	9
LIST OF APPENDICES	10
INTRODUCTION	10
1.1 Background and Context.....	11
1.2 Literature Review.....	12
1.3 Research Gap	15
2. OBJECTIVE	17
3. METHODOLOGY	17
4. HIGH-LEVEL SYSTEM ARCHITECTURE	23
5. USER REQUIREMENTS	25
Security Requirements	26
6. COMMERCIALIZATION PLAN	28
7. BUDGET AND JUSTIFICATION	29
8. WORK BREAKDOWN STRUCTURE (WBS)	31
9. GANTT CHART	32
10. REFERENCES LIST.....	34
11. APPENDICES.....	35

LIST OF TABLES

<i>Table 1 - Existing Systems and Limitations</i>	14
<i>Table 2- Risk & Mitigation</i>	15
<i>Table 3 Estimated Budget for System Implementation and Experimentation</i>	29
<i>Table 4 Task Distribution Among Team Members</i>	31
<i>Table 5 Project Timeline Alignment</i>	32
<i>Table 6 - Gantt Chart</i>	33

LIST OF FIGURES

<i>Figure 1- Methodology Workflow Diagram</i>	23
<i>Figure 2 -High-Level System Architecture Diagram</i>	24
<i>Figure 3 - System Workflow</i>	25
<i>Figure 4 - Users & Stakeholders</i>	27

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AST	Abstract Syntax Tree
CFG	Control Flow Graph
API	Application Programming Interface
JSON	JavaScript Object Notation
IDE	Integrated Development Environment
SDLC	Software Development Life Cycle
AI	Artificial Intelligence
ML	Machine Learning
DSR	Design Science Research
REST	Representational State Transfer
NFR	Non-Functional Requirement
FR	Functional Requirement
WBS	Work Breakdown Structure
CI/CD	Continuous Integration / Continuous Deployment
QA	Quality Assurance

LIST OF APPENDICES

Appendix	Title
Appendix A	Developer Survey Instrument for Evaluating the Code Analysis Agent
Appendix B	Detailed System Architecture Diagram of the Proposed System
Appendix C	Risk Analysis and Risk Mitigation Plan
Appendix D	Ethical Considerations and Research Ethics Compliance

1. INTRODUCTION

Software quality plays a crucial role in the success and sustainability of modern software systems. As software applications continue to grow in size and complexity, maintaining clean,

efficient, and well-structured code becomes increasingly challenging for development teams. Over time, software projects often accumulate structural issues in the codebase, commonly referred to as **code smells**, which may not immediately break functionality but can significantly affect maintainability, readability, and future development.

To address these challenges, developers rely on static code analysis tools such as **SonarQube** and **JDeodorant** to detect potential code quality issues and refactoring opportunities. These tools help identify problematic areas in the code, including duplicated logic, long methods, and poorly structured classes. However, while such tools are effective at detecting issues, they often provide large numbers of warnings without offering clear guidance on the most effective refactoring strategies.

As a result, developers must still spend significant time analyzing these reports and determining how to improve the codebase manually. This highlights the need for more intelligent systems that not only detect code smells but also assist developers by recommending suitable refactoring strategies.

Therefore, this research proposes the development of an intelligent analysis and recommendation system that enhances existing static code analysis approaches. The proposed system aims to support developers by identifying critical code quality issues and providing meaningful refactoring recommendations to improve maintainability and overall software quality.

1.1 Background and Context

In recent years, the rapid growth of software systems has significantly increased the complexity of maintaining high-quality codebases. Large-scale software projects often contain thousands of lines of code developed by multiple programmers over extended periods. As systems evolve, the codebase may accumulate various **code smells**, which are structural issues in the source code that indicate potential design problems. Although code smells do not necessarily cause immediate software failures, they negatively affect code maintainability, readability, scalability, and overall software quality.

One commonly observed issue in large codebases is the presence of **long methods**, **duplicate code**, **god classes**, and other design flaws that increase technical debt. These issues make it difficult for developers to understand, modify, and extend the software efficiently. Studies in software engineering indicate that poor code quality significantly increases maintenance costs, which typically account for more than **60–80% of the total software lifecycle cost**. As a result, identifying and refactoring problematic code segments has become a critical task in modern software development.

Software development teams often rely on automated static analysis tools to detect potential code quality issues. Tools such as **SonarQube** provide automated code inspection to identify bugs, vulnerabilities, and code smells. Similarly, tools like **JDeodorant** focus on detecting specific refactoring opportunities such as Extract Method, Move Method, or Extract Class. While these tools provide valuable insights, they still present several limitations. For example, many tools generate a large number of warnings without prioritizing which refactoring actions

should be performed first. In addition, developers often struggle to interpret the results and decide the most effective refactoring strategy to improve the codebase.

Another limitation of existing tools is that they primarily focus on detection rather than **providing intelligent recommendations for automated or semi-automated refactoring**. Developers must still manually analyze the reported issues and determine how to refactor the code efficiently. This process can be time-consuming and may require significant expertise in software design principles.

To address these limitations, this research proposes a system that enhances traditional static code analysis by integrating advanced analysis techniques to detect and recommend optimal refactoring strategies. The proposed solution will analyze source code using tools such as SonarQube and JDeodorant and apply additional analysis to prioritize and recommend appropriate refactoring actions. The system aims to assist developers in improving code quality more effectively by providing clearer insights and practical suggestions for code improvement. The proposed system will consist of several key components, including a **code analysis module**, a **refactoring recommendation engine**, and a **visual interface for developers**. The code analysis module will extract code metrics and detect potential design issues. The recommendation engine will evaluate the severity and impact of detected code smells and suggest suitable refactoring methods. Finally, the user interface will present the results in an intuitive manner, enabling developers to easily understand and apply the recommendations.

This research also aligns with global efforts to promote sustainable and efficient technological development. In particular, it supports the objectives of **United Nations Sustainable Development Goal SDG 9: Industry, Innovation and Infrastructure**, which encourages innovation and the development of reliable and sustainable infrastructure. By improving software quality and maintainability, this research contributes to building more reliable digital systems that support modern industries.

Furthermore, the research aligns with the **Software Engineering and Intelligent Systems research cluster**, as it focuses on improving automated software quality analysis using intelligent techniques. The outcome of this research has the potential to support developers, software companies, and educational institutions by providing a more effective approach to code quality improvement and refactoring assistance.

1.2 Literature Review

Software maintainability, structural integrity, and automated refactoring have become central research areas in modern software engineering. As systems scale in complexity, preserving architectural quality and minimizing technical debt become increasingly challenging. Several static analysis tools and research contributions have addressed code smell detection, architectural degradation, and quality reporting. However, most existing systems focus on rule-based detection or developer-oriented dashboards rather than generating structured architectural intelligence optimized for automated refactoring agents.

One of the most widely adopted industrial tools for continuous quality inspection is SonarQube [1]. SonarQube performs rule-based static analysis to detect bugs, vulnerabilities, and code smells while providing maintainability, reliability, and security ratings. It also estimates technical debt based on remediation effort. Although effective for identifying localized issues,

SonarQube primarily generates human-readable dashboards and issue lists. Its output structure is not inherently designed for downstream AI-based refactoring planning.

Similarly, tools such as PMD [2] and SpotBugs [3] perform static rule-based inspections. PMD detects suboptimal coding patterns and complexity issues, while SpotBugs analyzes compiled bytecode to identify potential defects. These tools focus mainly on code-level issues and lack comprehensive architectural modeling or structured intelligence representation suitable for multi-agent analysis systems.

Architectural-level static analysis tools such as JArchitect [4] and CppDepend [5] extend beyond traditional code inspection by providing dependency graphs, coupling metrics, and architectural constraint validation. These tools allow developers to define custom rules and analyze structural relationships. However, they primarily serve as decision-support systems for developers rather than generating structured machine-readable architectural knowledge intended for autonomous agents.

Research on automated refactoring and smell detection was significantly influenced by Fowler’s refactoring principles, which formalized the concept of code smells as indicators of deeper design problems. Building upon this foundation, JDeodorant [6] introduced automated detection of object-oriented smells such as God Class, Feature Envy, Long Method, and Type Checking. It further provides semi-automated refactoring suggestions within an integrated development environment. While JDeodorant performs structural dependency analysis, its scope remains limited to predefined smell types and does not formalize a generalized intelligence schema for agent-based planning systems.

DesigniteJava [7] further expands smell detection to include implementation, design, and architectural smells in Java systems. It provides hierarchical categorization and layered structural insights. Nevertheless, its reporting mechanisms are largely visualization-oriented and not explicitly designed for structured inter-agent communication.

Recent systematic literature reviews provide broader perspectives on detection methodologies. A comprehensive systematic review on software code smells [8] categorizes detection techniques into metric-based, rule-based, search-based, and machine learning-based approaches. The review highlights that most traditional techniques rely on threshold-based metric evaluation, which may result in context-insensitive detection and false positives. Another systematic review [9] emphasizes the increasing use of machine learning and deep learning models for smell detection. These approaches leverage abstract syntax trees, semantic embeddings, and graph-based representations to improve classification accuracy. However, most outputs remain classification labels rather than structured architectural representations suitable for automated reasoning systems.

Recent deep learning-based approaches such as EnseSmells [10] combine ensemble learning with programming language models to improve automated smell detection accuracy. Similarly, PyExamine [11] introduces multi-layer smell detection for Python systems, identifying code-level, design-level, and architectural-level smells. While these approaches demonstrate improved detection precision, they primarily focus on predictive accuracy rather than structured quality intelligence modeling for multi-agent architectures.

Architectural smell research further explores structural degradation at higher abstraction levels. The empirical study on the relationship between architectural and code smells [12] demonstrates that architectural smells represent distinct structural phenomena that cannot be fully captured through code-level metrics alone. Additionally, the work on anticipating architectural smells using link prediction techniques [13] highlights the importance of dependency graph modeling and structural network analysis. These findings reinforce the significance of graph-based architectural representations in detecting structural weaknesses.

Technical debt modeling also plays a significant role in quality assessment. Industrial platforms such as SonarQube [1] estimate remediation effort and provide debt ratios, but often lack contextual architectural encoding. Research suggests that effective automated planning systems require structured outputs incorporating entity relationships, severity levels, confidence metrics, and smell correlations.

The reviewed literature demonstrates that existing tools and approaches effectively detect code and architectural smells but primarily focus on developer-facing reports or classification outputs. There remains a research gap in systems that integrate structural metrics, smell correlation, and architectural rule validation into a unified machine-readable intelligence schema optimized for automated refactoring planning agents.

Therefore, the proposed structured code quality analysis agent distinguishes itself by emphasizing architectural knowledge modeling and structured reporting rather than solely detection accuracy. By generating machine-readable outputs (e.g., JSON-based structured quality reports) alongside human-readable summaries, the system supports both automated refactoring agents and developer interpretation. This positioning addresses a critical gap between traditional static analysis tools and emerging multi-agent refactoring architectures.

Table 1 - Existing Systems and Limitations

Existing System / Tool	Limitations (Considering Our Proposed System)
SonarQube	Produces large numbers of warnings but does not provide intelligent decision support or structured refactoring planning
PMD	Focuses mainly on rule checking and does not prioritize issues based on maintainability impact
Checkstyle	Limited to style and formatting checks and does not analyze deeper design or architectural problems
JDeodorant	Provides refactoring suggestions but lacks automated decision-making and structured planning for refactoring tasks
CodeScene	Provides insights into code complexity but does not guide developers with actionable refactoring plans

1.3 Research Gap

Despite the availability of modern static code analysis tools, there are still several limitations in how software quality issues are detected, interpreted, and resolved. Tools such as **SonarQube** and **JDeodorant** are widely used in the industry to identify code smells, design flaws, and refactoring opportunities. These tools can automatically scan large codebases and generate reports highlighting potential issues. However, they primarily focus on **detecting problems rather than guiding developers toward the most effective solutions**.

One major limitation of current tools is the **lack of intelligent prioritization of detected code smells**. In large-scale software projects, static analysis tools often generate hundreds or even thousands of warnings. Developers must manually analyze these reports to determine which issues are critical and which refactoring strategies should be applied. This process can be time-consuming and may lead to inconsistent decisions depending on the developer's experience.

Another limitation is that many existing tools **focus on individual types of code smells rather than providing a holistic analysis of the entire codebase**. For example, some tools specialize in identifying specific refactoring opportunities, but they do not consider the broader impact of refactoring decisions on the system architecture or maintainability.

Furthermore, current solutions provide limited assistance in **bridging the gap between code smell detection and practical refactoring implementation**. Developers often receive warnings about design issues but are not provided with clear recommendations on how to refactor the code effectively. As a result, valuable insights generated by these tools may remain underutilized.

Therefore, there is a need for a more advanced approach that goes beyond traditional static analysis by integrating **intelligent analysis and recommendation mechanisms**. The proposed research aims to address this gap by developing a system that not only detects code smells using existing analysis tools but also provides **prioritized and meaningful refactoring recommendations** to assist developers in improving software quality more efficiently.

By introducing an intelligent recommendation framework that integrates multiple analysis techniques, this research seeks to enhance the usability and effectiveness of code quality tools. This approach represents a novel contribution toward supporting developers in making more informed refactoring decisions and improving the long-term maintainability of software systems.

1.4 Risk & Mitigation

Table 2- Risk & Mitigation

Risk	Description	Mitigation Strategy
Inaccurate Refactoring Decisions	The system may incorrectly identify when refactoring is required	Use validated code quality metrics and test the decision model with multiple datasets

Data Availability Issues	Difficulty obtaining suitable real-world code quality reports for experiments	Use open-source projects from GitHub and publicly available datasets
Integration Challenges	Difficulty integrating static analysis tools with the proposed system	Use standard report formats such as JSON or XML for easier integration
Evaluation Complexity	Measuring actual improvements in software quality may be challenging	Use multiple evaluation metrics such as maintainability index, complexity, and code smell reduction
Performance Overhead	Processing large codebases may require significant computational resources	Optimize analysis algorithms and test with scalable datasets

2. OBJECTIVE

2.1 Main Objective

To develop an intelligent software analysis system that detects code quality issues and provides prioritized machine readable report to improve the maintainability, readability, and overall quality of software systems.

2.2 Specific Objectives

1. **Repository Code Analysis using Open-Source Tools**

Integrate open-source static analysis tools such as SonarQube to automatically analyze software repositories and detect common code quality issues including bugs, code smells, and maintainability violations.

2. **Rule-Based Code Smell Detection**

Design and implement a rule-based analysis module that detects coding standard violations and common code smells using predefined rules such as method length limits, class size thresholds, and duplication patterns.

3. **API-Based Integration of External Analysis Tools**

Utilize the REST API capabilities of tools like SonarQube to programmatically retrieve analysis results and integrate them into the proposed system for further processing and decision making.

4. **Refactoring Opportunity Identification**

Detect potential refactoring opportunities using outputs from tools such as JDeodorant together with rule-based detection mechanisms to identify structural improvement opportunities in the codebase.

5. **Issue Aggregation and Prioritization Mechanism**

Develop a processing component that consolidates detected issues from both tool-based analysis and rule-based detection, and prioritizes them based on severity, frequency, and potential impact on software maintainability.

6. **Structured Output Generation for Multi-Agent Processing**

Generate structured analysis outputs (e.g., JSON reports) that summarize detected code smells, rule violations, and recommended refactoring actions so that they can be consumed by other agents in the automated software improvement pipeline.

3. METHODOLOGY

3.1 Research Approach

This research adopts a **Design Science Research (DSR) approach** combined with experimental evaluation. Design Science Research is widely used in software engineering studies where the objective is to **design and evaluate a technological artifact that addresses a real-world problem**.

The artifact developed in this research is an **Intelligent Code Analysis Agent** that detects software quality issues in source code repositories by integrating static analysis results with a rule-based detection system.

The research process consists of the following stages:

Problem Identification

Software developers often struggle to detect maintainability issues such as code smells, excessive complexity, and coding standard violations in large codebases. While static analysis tools exist, they often require manual interpretation and may not cover certain rule-based coding practices.

Solution Design

To address this challenge, a software agent is designed to automatically analyze repositories, combine static analysis outputs, and apply rule-based detection to identify additional quality issues.

Artifact Development

The agent is implemented using Python and integrated with static analysis tools to automatically process source code repositories.

Experimental Evaluation

The system is evaluated using open-source repositories to measure its effectiveness in detecting code quality issues and improving maintainability insights.

This approach ensures that the research contributes both a **practical tool and empirical evaluation results**.

3.2 Technical Methods and Algorithms

The proposed component functions as an **automated code analysis agent** that examines source code repositories and identifies software quality issues using a hybrid analysis method combining **static analysis and rule-based detection**.

The methodology consists of the following stages.

Step 1 – Repository Acquisition

The system retrieves the source code repository from version control platforms such as GitHub.

The repository ingestion module performs the following tasks:

- Clone the repository
- Extract the project directory structure
- Identify programming language
- Collect all relevant source code files

These files are then prepared for automated analysis.

Step 2 – Integrated Static and Rule-Based Code Analysis

In this stage, the system performs a **combined analysis process** that integrates static analysis outputs with internally defined rule-based detection.

The static analysis component uses SonarQube to scan the source code repository.

SonarQube detects several software quality issues, including:

- Code smells
- Maintainability issues
- bugs
- code duplication
- complexity metrics

The system retrieves these analysis results through the **SonarQube REST API**.

At the same time, the agent performs **rule-based analysis using Python** to detect additional coding standard violations and structural issues.

This analysis uses **Abstract Syntax Tree (AST) parsing** to examine the structure of the source code.

Example detection rules include:

- Long Method Detection
*IF method_length > predefined_threshold
THEN detect Long Method smell*
- Large Class Detection
*IF number_of_methods_in_class > threshold
THEN detect Large Class smell*
- High Cyclomatic Complexity Detection
*IF cyclomatic_complexity > threshold
THEN mark as complex method*

These rule-based checks allow the system to identify issues that may not be explicitly reported by static analysis tools.

The outputs from SonarQube and the rule-based detection engine are then merged into a unified dataset.

Step 3 – Issue Aggregation

The system aggregates all detected issues from both the static analysis and rule-based detection modules.

During this stage:

- Duplicate issues are located
- Issue types are categorized
- Source code locations are recorded
- Severity levels are assigned
-

This produces a consolidated list of software quality issues for the analyzed repository.

Step 4 – Issue Prioritization

The detected issues are prioritized using a scoring mechanism based on:

- Severity level
- Complexity metrics
- Frequency of occurrence

This prioritization helps identify the most critical issues that affect software maintainability.

Step 5 – Structured Output Generation

The final analysis results are converted into structured outputs such as JSON objects.

Each issue record contains:

- Issue type
- File location
- Severity level
- Complexity metrics

These structured outputs can be consumed by other agents in the system pipeline or displayed in a developer interface.

3.3 Tools and Platforms with Justification

The implementation uses open-source technologies that support automation, analysis, and integration.

Programming Language.

Python is used as the primary programming language due to its flexibility, strong ecosystem of analysis libraries, and ability to integrate with external APIs.

Repository Platform

GitHub is used to store and retrieve software repositories and manage the open-source development process.

Static Analysis Tool

SonarQube is used for automated static code analysis because it provides comprehensive software quality metrics and supports integration through REST APIs.

Code Parsing Libraries

Python libraries such as AST and Radon are used to analyze code structures and calculate complexity metrics required for rule-based detection.

These tools are selected because they are **widely used in software engineering research and freely available for academic development.**

3.4 Data Collection and Ethical Considerations

The dataset used in this research consists of publicly available software repositories. Source code data is collected from open-source platforms such as GitHub.

The collected data includes:

- Source code files
- Software quality metrics
- Code smell reports
- Complexity measurements

Ethical considerations include the following:

- Only publicly accessible repositories are used.
- No private or confidential codebases are analyzed.

- Repository authors are properly acknowledged if their projects are used for evaluation.
- Analysis results are used strictly for research purposes.

This ensures that the study follows ethical guidelines for software engineering research.

3.5 Validation Strategy and Performance Metrics

The proposed agent will be evaluated using an experimental validation strategy.

The validation process includes:

- Selecting multiple open-source repositories.
- Running the proposed analysis agent on these repositories.
- Collecting detected code smell and complexity metrics.
- Comparing the analysis results with baseline static analysis outputs.

The following performance metrics are used for evaluation.

Code Smell Detection Coverage

Measures the number of code quality issues detected by the system.

Detection Accuracy

Evaluates whether the rule-based system correctly identifies real software quality problems.

Processing Time

Measures the time required for the agent to analyze repositories of different sizes.

Complexity Identification Accuracy

Evaluates the ability of the system to correctly detect highly complex methods and classes.

These metrics help determine the effectiveness of the proposed solution in supporting software quality improvement.

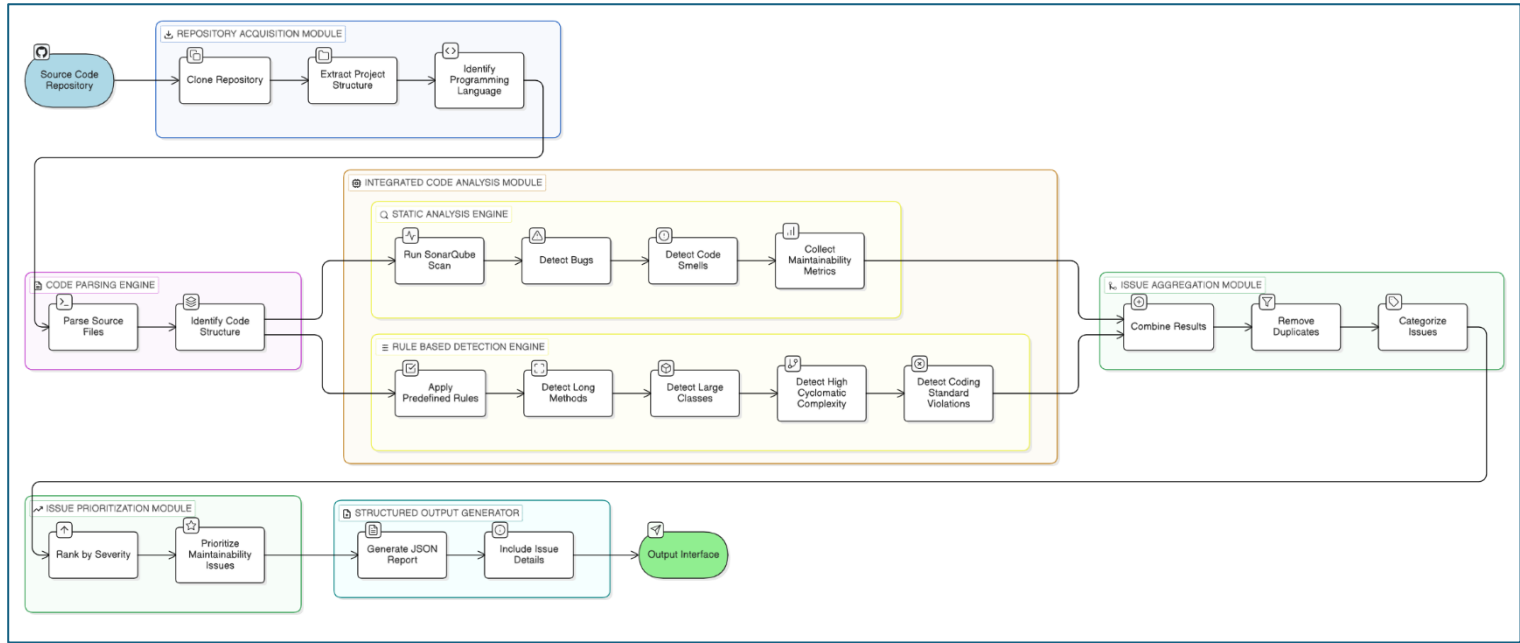


Figure 1- Methodology Workflow Diagram

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of the Proposed Solution

The proposed system is designed as a **multi-agent software analysis and refactoring support platform** that assists developers in identifying, prioritizing, and improving software quality issues in large codebases. The system processes source code repositories and applies automated analysis techniques to detect code smells, coding standard violations, and potential refactoring opportunities.

The architecture consists of **four intelligent agents**, where each agent performs a specialized task within the automated software quality improvement pipeline. The first agent focuses on **code analysis and rule-based smell detection**, while the remaining agents perform further tasks such as refactoring recommendation, transformation, and validation.

The system integrates open-source analysis tool such as **SonarQube** together with a **custom rule-based analysis module**. This approach allows the system to combine existing static analysis capabilities with additional intelligent processing to produce structured outputs that can be consumed by other agents in the pipeline.

The goal of this architecture is to create a **scalable and modular analysis framework** that supports automated decision-making in software maintenance and refactoring processes.

4.2 High-Level System Architecture Diagram

[High-Level System Architecture Diagram.png](#)

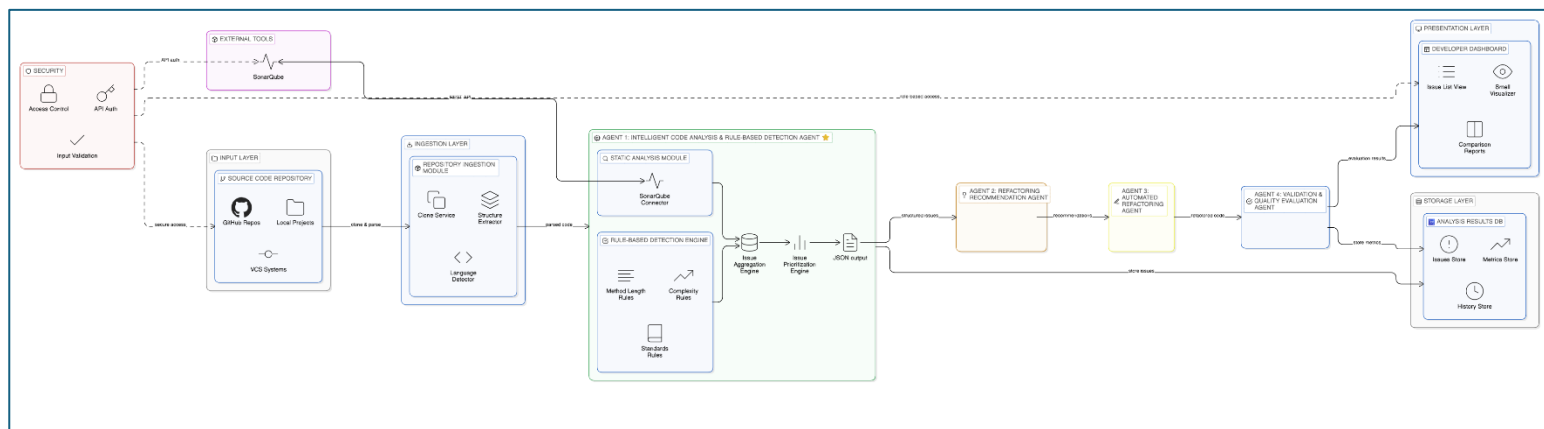


Figure 2 -High-Level System Architecture Diagram

Repository Input Module

Retrieves source code repositories from local directories or version control systems such as GitHub.

Static Analysis Integration Module

Uses APIs from tools like **SonarQube** to obtain detailed code quality reports including code smells, bugs, and maintainability metrics.

Rule-Based Analysis Engine

Apply predefined rules to detect coding standard violations and structural issues such as long methods, large classes, or duplicated logic.

Issue Aggregation Module

Combines results from multiple sources and organizes them into a unified representation of code quality problems.

Structured Output Generator

Generate machine-readable output (JSON or API responses) that can be used by the next agents in the system.

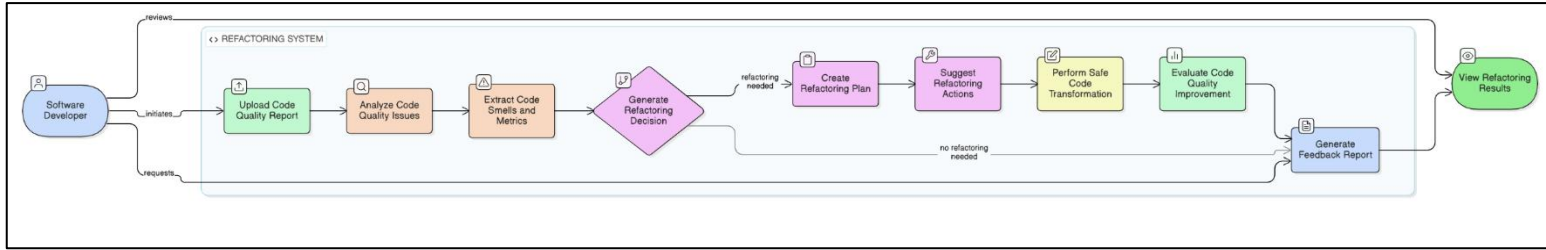


Figure 3 - System Workflow

5. USER REQUIREMENTS

5.1. Stakeholder Identification

The proposed system involves several stakeholders who interact with the platform directly or benefit from the results generated by the analysis agents.

Software Developers

Software developers are the primary users of the system. They use the platform to analyze their source code repositories, identify code smells, detect coding standard violations, and receive prioritized refactoring recommendations to improve software maintainability.

Software Development Teams / Organizations

Development teams can use the system during the software development lifecycle to monitor code quality, reduce technical debt, and maintain coding standards across large collaborative projects.

Software Quality Assurance Engineers

Quality assurance engineers can use the analysis results to evaluate the structural quality of software systems and ensure that coding standards and maintainability metrics meet organizational requirements.

Project Maintainers of Open-Source Repositories

Maintainers of open-source projects can use the system to analyze repository quality and identify improvement opportunities in large codebases.

Researchers and Students

Researchers and students can use the system as a learning tool to understand software design issues, refactoring opportunities, and code quality metrics.

5.2. Functional Requirements

The following functional requirements define the key operations that the system must support.

FR1 – Repository Input

The system shall allow users to upload or connect a source code repository from local directories or version control platforms such as GitHub.

FR2 – Source Code Analysis

The system shall analyze the uploaded repository using static code analysis tools such as SonarQube to detect code quality issues including bugs, vulnerabilities, and code smells.

FR3 – Rule-Based Code Smell Detection

The system shall apply predefined rules to detect coding standard violations and structural issues such as long methods, large classes, and high complexity levels.

FR4 – Issue Aggregation

The system shall combine results from multiple analysis tools and rule-based detection modules to produce a unified list of detected code quality issues.

FR5 – Issue Prioritization

The system shall prioritize detected issues based on severity, complexity, and potential impact on software maintainability.

FR6 – Structured Output Generation

The system shall generate structured analysis outputs (e.g., JSON or API responses, Graphs) that can be processed by other agents in the automated refactoring pipeline.

FR7 – Visualization of Analysis Results

The system shall provide a dashboard or interface that displays detected code issues, severity levels, and suggested improvements.

5.3. Non-Functional Requirements

Non-functional requirements define the quality attributes that the system must satisfy.

Security Requirements**NFR1 – Secure Repository Access**

The system shall ensure secure access when connecting to external repositories by using authentication mechanisms.

NFR2 – Data Protection

The system shall protect analysis data and prevent unauthorized access to repository information.

Performance Requirements**NFR3 – Analysis Performance**

The system shall analyze medium-sized repositories within a reasonable processing time to ensure usability for developers.

NFR4 – Efficient Resource Usage

The system shall optimize processing resources when performing static code analysis and rule-based detection.

Usability Requirements

NFR5 – User-Friendly Interface

The system shall provide a clear and intuitive interface for developers to view code quality results and refactoring recommendations.

NFR6 – Clear Visualization of Issues

The system shall present detected code issues in a structured and easily understandable format.

Scalability Requirements

NFR7 – Multi-Repository Support

The system shall support the analysis of multiple repositories without affecting system performance.

NFR8 – Modular Agent Architecture

The system shall use a modular agent-based design that allows future expansion and integration with additional analysis tools.

Users & Stakeholders.png

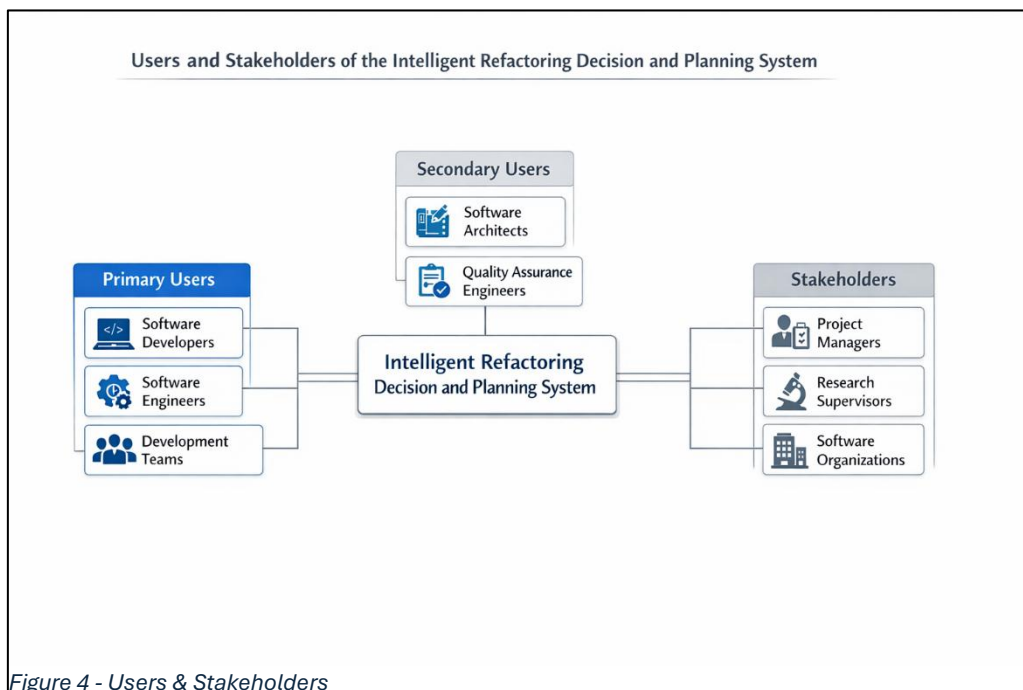


Figure 4 - Users & Stakeholders

6. COMMERCIALIZATION PLAN

The proposed Automated Multi-Agent Software Refactoring System has strong commercialization potential within the global software development and software maintenance industry. The primary target market includes software development companies, technology startups, and large enterprises that maintain complex software systems or legacy codebases. These organizations frequently struggle with technical debt, poor code quality, and the increasing complexity of maintaining large software systems over time. Development teams that actively use AI-assisted coding tools such as GitHub Copilot or other large language model-based coding assistants represent a particularly relevant customer segment, as these tools often accelerate code generation without ensuring long-term architectural quality. The main users of the system would include software developers, software architects, DevOps engineers, and quality assurance teams responsible for maintaining and improving code quality.

The value proposition of the proposed system lies in its ability to provide an automated and intelligent refactoring workflow that analyzes code quality issues and generates structured refactoring plans before performing code transformations. Unlike traditional static analysis tools that only detect code smells or provide isolated recommendations, the proposed multi-agent framework integrates code analysis, strategic refactoring planning, safe code transformation, and validation into a unified pipeline. The Refactoring Decision and Planning Agent (RDP Agent) plays a key role in this architecture by generating dependency-aware refactoring plans that guide the transformation process and ensure that refactoring actions are applied in an optimal sequence.

From a commercialization perspective, the system can be delivered as a developer productivity platform integrated into modern software development environments. The system could be deployed as an integrated development environment (IDE) plugin, a cloud-based code quality analysis service, or an enterprise software maintenance platform integrated with version control systems such as GitHub or GitLab. A Software-as-a-Service (SaaS) revenue model could be adopted, where development teams subscribe to the platform on a monthly or annual basis to analyze repositories, generate refactoring plans, and improve software quality across projects.

The operational costs associated with commercial deployment would primarily include cloud infrastructure for code analysis and machine learning processing, platform maintenance and updates, and technical support services for enterprise users. A tiered pricing strategy could be implemented to accommodate different types of customers. For example, smaller development teams and startups could access a basic version of the platform with limited analysis capabilities, while larger enterprises could purchase premium plans that support large-scale repository analysis, advanced refactoring planning, integration with DevOps pipelines, and team collaboration features.

The system maintains a competitive advantage by providing a planning-driven automated refactoring solution that considers dependencies, architectural constraints, and expected quality improvements before executing transformations. While many existing tools focus only on detecting code smells or performing individual refactoring actions, the proposed framework

introduces an intelligent planning layer that coordinates multiple refactoring operations within a structured workflow. This approach enables safer and more effective large-scale refactoring of complex software systems.

In terms of intellectual property, the core innovation of the system lies in the multi-agent architecture and the planning mechanisms used to generate structured refactoring strategies. The algorithms, decision models, and planning architecture developed during this research can be protected through software copyright or intellectual property registration. Additionally, any proprietary machine learning models and training datasets used for predicting refactoring suitability may provide further competitive advantages for a commercial implementation of the system.

7. BUDGET AND JUSTIFICATION

The proposed research mainly requires cloud computing resources to run experiments, store datasets, and host the prototype system. Most software tools used in the project such as Python libraries, SonarQube Community Edition, and open-source frameworks are freely available. Therefore, the required budget mainly focuses on minimal cloud infrastructure and hosting services.

Table 3 Estimated Budget for System Implementation and Experimentation

Resource	Purpose	Estimated Cost (LKR)
Cloud Compute Instance (AWS EC2 / DigitalOcean Droplet)	Running code analysis agents and refactoring planning experiments	LKR 7,798.75 / month
Cloud Storage (AWS S3 / Google Cloud Storage)	Storing datasets, repositories, and generated refactoring plans	LKR 935.85 / month
Small Database Service (AWS RDS / Managed PostgreSQL)	Storing software metrics, code quality reports, and experiment results	LKR 3,119.50 / month
Web Hosting / Server Hosting	Hosting the prototype system interface and API	LKR 3,119.50 / month
Domain and Backup Storage	Domain registration and backup of experiment data	LKR 623.90 / month

Total Estimated Budget

Total Cost: 15,597.50 LKR per month

Cloud Compute Instance

A cloud server will be used to run the multi-agent refactoring system, perform static code analysis, and execute experiments on open-source software repositories.

Cloud Storage

Cloud storage services will store datasets, software repositories, experiment results, and generated refactoring plans produced during evaluation.

Database Service

A lightweight database will store structured data such as software metrics, dependency relationships, and refactoring planning outputs.

Web Hosting

A basic hosting service will deploy the prototype system interface, allowing users to submit code quality reports and obtain refactoring plans generated by the planning agent.

Domain and Backup Storage

A small budget is allocated for domain registration and backup storage to ensure reliability and accessibility of the experimental system.

8. WORK BREAKDOWN STRUCTURE (WBS)

The project work is divided among the four team members according to the specialized agents in the multi-agent architecture. Each member is responsible for the design, implementation, and evaluation of a specific agent while collaborating on common tasks such as system integration, testing, and documentation.

Table 4 Task Distribution Among Team Members

Team Member	Component / Agent	Key Responsibilities
Member 1	Code Understanding Agent	Analyzing the source code structure, identifying code smells, generating complexity metrics, and producing structured quality reports.
Member 2	Refactoring Planning Agent	Generating intelligent refactoring strategies based on analysis results, prioritizing refactoring actions, and preparing machine-executable refactoring plans.
Member 3	Safe Code Transformation & Validation Agent	Applying AST-based code transformations, validating syntactic and semantic correctness, performing behavioral validation, and handling rollback when necessary.
Member 4	Developer Interaction & Workflow Orchestration Agent	Managing the interaction between developers and the system, coordinating workflow among agents, presenting refactoring plans and results through a dashboard or IDE interface.

Although each member is responsible for a specific component, several tasks are performed collaboratively to ensure system consistency and integration. These shared tasks include system architecture design, integration of all agents into a unified platform, testing and validation of the complete system, evaluation of experimental results, and preparation of research documentation.

The major development activities of the project are organized into phases that align with the research timeline. The project begins with literature review and system design activities, followed by implementation of the individual agents. After the development stage, the agents are integrated into a unified system and evaluated using selected legacy code repositories. Finally, documentation and final reporting activities are completed.

Table 5 Project Timeline Alignment

Project Phase	Activities	Timeline
Phase 1	Literature review, problem analysis, and research proposal preparation	Month 1 – Month 2
Phase 2	System architecture design and agent specification	Month 3
Phase 3	Implementation of Code Understanding Agent	Month 4 – Month 5
Phase 4	Implementation of Refactoring Planning Agent	Month 4 – Month 5
Phase 5	Implementation of Safe Transformation & Validation Agent	Month 5 – Month 6
Phase 6	Implementation of Developer Interaction & Workflow Orchestration Agent	Month 5 – Month 6
Phase 7	Integration of agents and system testing	Month 7
Phase 8	Experimental evaluation and performance analysis	Month 8
Phase 9	Final documentation, report writing, and presentation preparation	Month 9

The workload distribution among team members is carefully balanced so that each member contributes equally to the development of the proposed system. Each agent requires comparable levels of research, design, and implementation effort. In addition, shared activities such as system integration, testing, evaluation, and documentation are carried out collaboratively by all members. This balanced distribution of tasks ensures efficient progress of the project while allowing each team member to develop expertise in a specific component of the multi-agent architecture.

9. GANTT CHART

The project is organized into multiple phases including literature review, system design, implementation of individual agents, system integration, evaluation, and final documentation. The schedule ensures that each stage of the project is completed within the academic timeline while allowing sufficient time for development, testing, and analysis.

Table 6 - Gantt Chart

Project Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6	Month 7	Month 8	Month 9
Literature Review	●	●							
Problem Identification	●	●							
System Architecture Design		●	●						
Research Proposal Preparation	●	●							
Development of Code Understanding Agent			●	●					
Development of Refactoring Planning Agent			●	●					
Development of Transformation & Validation Agent				●	●				
Development of Workflow Orchestration Agent				●	●				
System Integration					●	●			
Testing and Validation						●	●		
Experimental Evaluation							●	●	
Final Report Writing							●	●	●
Final Presentation Preparation								●	●

10. REFERENCES LIST

- [1] SonarSource, “SonarQube: Fight AI Slop & Verify AI Code | SonAR,” *Sonar*.
<https://www.sonarsource.com/products/sonarqube/>
- [2] Pmd, “GitHub - pmd/pmd: An extensible multilanguage static code analyzer.,” *GitHub*.
<https://github.com/pmd/pmd>
- [3] “Intro to SpotBugs,” *Baeldung*, Jul. 16, 2024. <https://www.baeldung.com/spotbugs-detect-bugs-code> (accessed Mar. 05, 2026).
- [4] JArchitect, <https://www.jarchitect.com>, “JArchitect :: Achieve Higher Java code quality.”
<https://www.jarchitect.com/GettingStarted> (accessed Mar. 05, 2026).
- [5] CoderGears, <https://www.codergears.com/home>, “CoderGears :: A complete suite of static analysis and code quality tools.” <https://www.codergears.com/home>
- [6] *Identification of Move Method Refactoring Opportunities*. [Online]. Available:
https://users.encs.concordia.ca/~nikolaos/publications/TSE_2009.pdf
- [7] “Google Scholar.”
https://scholar.google.com/citations?view_op=view_citation&hl=en&user=VHjAIe8AAAAJ&citation_for_view=VHjAIe8AAAAJ:TFP_iSt0sucC
- [8] M. G. Al-Obeidallah and D. Al-Fraihat, “A systematic review on software code smells,” *International Journal of Power Electronics and Drive Systems/International Journal of Electrical and Computer Engineering*, vol. 15, no. 3, p. 3010, May 2025, doi: 10.11591/ijece.v15i3.pp3010-3027.
- [9] T. T. Khoei, H. O. Slimane, and N. Kaabouch, “Deep learning: systematic review, models, challenges, and research directions,” *Neural Computing and Applications*, vol. 35, no. 31, pp. 23103–23124, Sep. 2023, doi: 10.1007/s00521-023-08957-4.
- [10] A. Ho, A. M. T. Bui, P. T. Nguyen, D. S. Amleto, and B. Le, “EnseSmells: Deep ensemble and programming language models for automated code smells detection,” *arXiv.org*, Feb. 07, 2025. <https://arxiv.org/abs/2502.05012>
- [11] “PyExamine: A Comprehensive, Un-Opinionated smell detection tool for Python.”
<https://arxiv.org/html/2501.18327v1>
- [12] *Anempirical investigation of the impact of architectural smells on software maintainability*. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1943772/FULLTEXT01.pdf>
- [13] *Towards Anticipation of Architectural Smells using Link Prediction Techniques*. [Online]. Available: <https://arxiv.org/pdf/1808.06362v1>

11.APPENDICES

Appendix A – Survey Instrument (Developer Feedback Questionnaire)

To evaluate the usefulness and practicality of the proposed automated code analysis agent, a short survey will be conducted among software developers and software engineering students. The survey aims to collect feedback regarding usability, usefulness, and accuracy of the system.

[Developer Feedback Questionnaire - Google Forms](#)

Survey Questions

1. How often do you use automated tools to analyze code quality?
 - Always
 - Often
 - Sometimes
 - Rarely
 - Never
2. Which tools do you currently use for static code analysis?
 - SonarQube
 - ESLint
 - PMD
 - Checkstyle
 - None
3. How difficult is it to manually identify code smells in large repositories?
 - Very Difficult
 - Difficult
 - Neutral
 - Easy
4. How useful would an automated code analysis agent be for your development workflow?
 - Very Useful
 - Useful
 - Neutral

- Not Useful
- 5. Would you prefer a tool that provides structured analysis output (JSON) for integration with other systems?
 - Yes
 - No
- 6. What additional features would you expect from such a tool?
 - Code smell detection
 - Refactoring suggestions
 - Code complexity visualization
 - CI/CD integration
- 7. Rate the usefulness of automated rule-based code analysis.
 - Very Useful
 - Useful
 - Neutral
 - Not Useful

The responses from the survey will help evaluate the **usability and practical relevance of the system**.

Appendix B – Detailed System Architecture Diagram

This appendix includes a detailed system architecture diagram illustrating the interaction between the system components.

System Components

The architecture of the proposed system consists of the following components:

1. **Repository Input Module**
 - Accepts repository URLs or uploaded project files.
2. **Repository Processor**
 - Downloads and prepares source code for analysis.
3. **Static Analysis Engine**
 - Performs automated code analysis using tools such as SonarQube.
4. **Rule-Based Code Smell Detector**

- Applies predefined rules to detect common software quality issues.

5. Issue Aggregation Module

- Combines issues detected from multiple analysis sources.

6. Prioritization Engine

- Ranks issues based on severity and impact.

7. Output Generator

- Generates structured output results in JSON format.

Appendix C – Risk Analysis

Risk	Description	Impact	Mitigation
Tool Integration Issues	Difficulty integrating external analysis tools	Medium	Use stable APIs and open-source libraries
Large Repository Processing	Analysis of large repositories may require more processing time	Medium	Optimize code processing techniques
False Positive Results	Some code smell detections may be inaccurate	Medium	Adjust rule thresholds and validate results
Data Handling Errors	Errors in parsing or processing repository files	Low	Implement error handling and logging
Limited Dataset for Evaluation	Limited number of repositories used for testing	Low	Use multiple open-source repositories

Appendix D – Ethical Considerations

The research project follows ethical guidelines for academic research.

Key ethical considerations include:

1. Use of Public Repositories

- Only publicly available repositories will be used for analysis.

2. No Personal Data Collection

- The system analyzes source code and does not collect personal user data.

3. Responsible Use of Open-Source Tools

- All external tools and libraries will be used in compliance with their respective licenses.

4. Transparency in Results

- Analysis results will be presented accurately without manipulation.

5. Participant Consent (if survey is conducted)

- Survey participants will be informed about the purpose of the research before participation.